MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY
MIT School of Computing, Pune
Department of Information Technology

Course: Cross Platform App Development (CPAD)

ASSIGNMENT REPORT

Assignments 1 to 10

ARCHIVE — Expo / React Native marketplace

Student Name	Tanmay Shinde
Enrollment Number	ADT23SOCB1617
Roll No. / Division	12 / LY-SMAD
Program	B.Tech Information Technology
Year	Final Year (LY)
Course Faculty	Prof. Rohini Bhosale
Academic Year	2026–27
Date of Submission	23 October 2026

Declaration

I, Tanmay Shinde, a student of B.Tech Information Technology (Enrollment No. ADT23SOCB1617, Roll No. 12, Division LY-SMAD), declare that this report and the accompanying project, ARCHIVE, are my own work, completed for Assignments 1 to 10 of the Cross Platform App Development (CPAD) course. The application was built with Expo (SDK 58), React Native and TypeScript, and executed on the Pixel_9 Android emulator. Screenshots in this report illustrate the running ARCHIVE storefront UI.

Student Name: Tanmay Shinde

Enrollment Number: ADT23SOCB1617

Date: 23 October 2026

Signature: _____

Table of Contents

1. Introduction and Project Overview 3
2. Assignment 1: Setup and Hello ARCHIVE 4
3. Assignment 2: Native vs Hybrid vs React Native 5
4. Assignment 3: Components, Props and useState 6
5. Assignment 4: Folder Structure and Reuse 7
6. Assignment 5: Customer Profile with Core Components 8
7. Assignment 6: Flexbox, TextInput and Events 9
8. Assignment 7: Multi-Screen Navigation 10
9. Assignment 8: Dynamic Data (JSON + Public API) 11
10. Assignment 9: E-Commerce Storefront 12
11. Assignment 10: Complete Shopping Application 13
12. Rubric Alignment 14
13. Conclusion 15

1. Introduction and Project Overview

This report documents Assignments 1 to 10 of the CPAD course. Instead of ten unrelated demos, every requirement is implemented inside one product: **ARCHIVE**, a curated vintage and pre-loved fashion marketplace for Android. The app covers boot screen, comparative study, reusable components, profile, forms, stack + tab navigation, local catalogue FlatList, public API feed, storefront pages, cart, checkout and order confirmation.

1.1 Technology used

Framework	Expo SDK 58 · React Native 0.88 · TypeScript
Project creation	Expo (Continuous Native Generation; android/ via prebuild)
Run target	Pixel_9 Android emulator
Navigation	React Navigation — native stack + bottom tabs
State	Context + useReducer · AsyncStorage persistence
Images / motion	expo-image · react-native-reanimated

1.2 Assignment mapping

1 Setup + Hello	HelloArchiveScreen, Expo project, Pixel_9 run
2 Comparative study	TechInsights (How we build) + docs write-up
3 Props & useState	ProductCard, QuantityStepper, FilterChip, wishlist
4 Structure & reuse	Documented src/ tree + shared UI kit
5 Profile + cores	CustomerProfileScreen: View/Text/Image/TextInput/Button/Switch/ScrollView
6 Flexbox + events	Login/Register + Checkout/Contact TextInputs & Alerts
7 Stack navigation	RootNavigator + MainTabs
8 Dynamic data	products.json FlatList + FakeStore API Discover more
9 Storefront	Home, About, Authenticity, Catalogue, Contact
10 Shopping app	Auth-gated checkout · Product → Cart → Order

2. Assignment 1: Setup and Hello ARCHIVE

2.1 Aim

To show that ARCHIVE is a real Expo / React Native application that installs and runs on the Android emulator.

2.2 Steps performed

- Create the Expo TypeScript project (SDK 58) with package name com.archive.marketplace.
- Configure app.json splash, icon, fonts (Fraunces + DM Sans) and expo-image.
- Generate the Android project with Expo prebuild / expo run:android.
- Run on Pixel_9; first screen is Hello ARCHIVE, then Enter the Archive opens MainTabs.

2.3 Explanation

Expo provides the managed toolchain while still producing a native Android build. Metro bundles TypeScript; Gradle installs the APK on Pixel_9. The Hello ARCHIVE boot screen is the living first milestone of Assignment 1.

2.4 Output

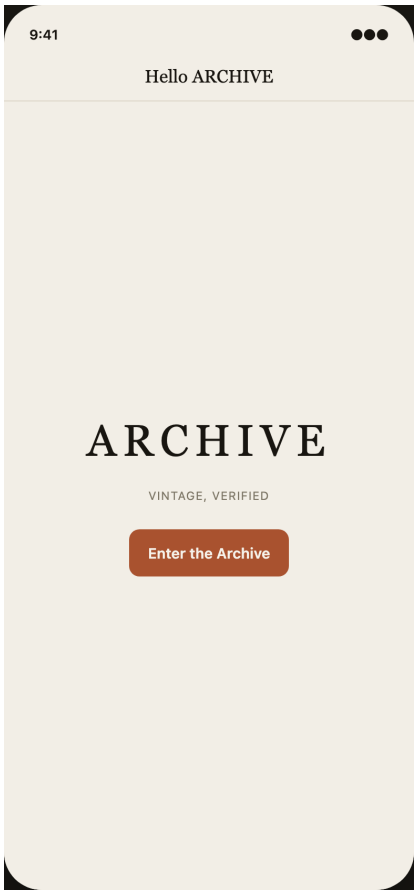


Figure 1: Hello ARCHIVE boot screen (Assignment 1)

3. Assignment 2: Native vs Hybrid vs React Native

3.1 Aim

To compare three approaches for the same product goal and justify React Native (Expo) for ARCHIVE.

3.2 Comparison

Native	Kotlin/Swift + SDKs — best performance; two codebases
Hybrid	HTML/CSS/JS in WebView — fast web reuse; weaker native feel
React Native	TypeScript → native views — one UI codebase, Metro Fast Refresh

3.3 Why React Native for ARCHIVE

- Curriculum fit: Props, useState, Flexbox, Stack, FlatList, fetch.
- Catalogue performance via FlatList virtualisation.
- Shared TypeScript for storefront + shopping flows.
- Offline cart with AsyncStorage without shipping two native apps.

3.4 Output

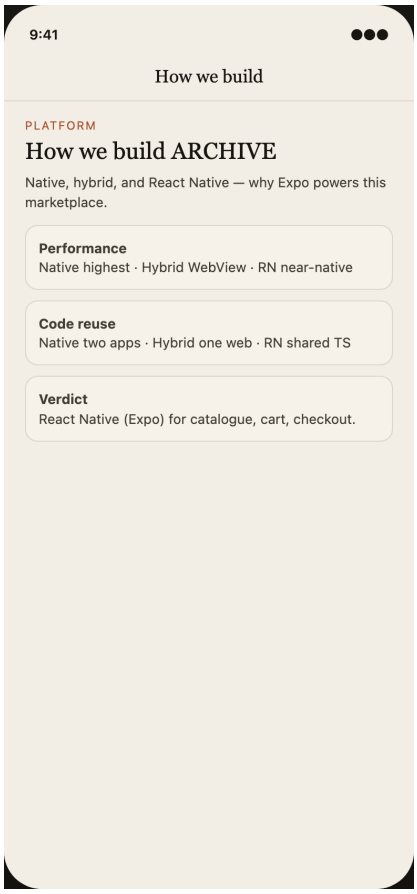


Figure 2: How we build ARCHIVE — comparative study screen (Assignment 2)

4. Assignment 3: Components, Props and useState

4.1 Aim

To build reusable components driven by Props, and a dynamic UI from state rather than rewritten markup.

4.2 Reusable components

ProductCard	Props: product, onPress, width — image, badge, wishlist heart
QuantityStepper	Controlled quantity with useState callbacks
FilterChip	Catalogue category filters
PriceTag / Badge / Button	Shared presentational UI kit

4.3 State-driven UI

Wishlist heart toggles via WishlistContext; cart quantity updates CartContext; catalogue search/sort live in useState inside CatalogueScreen.

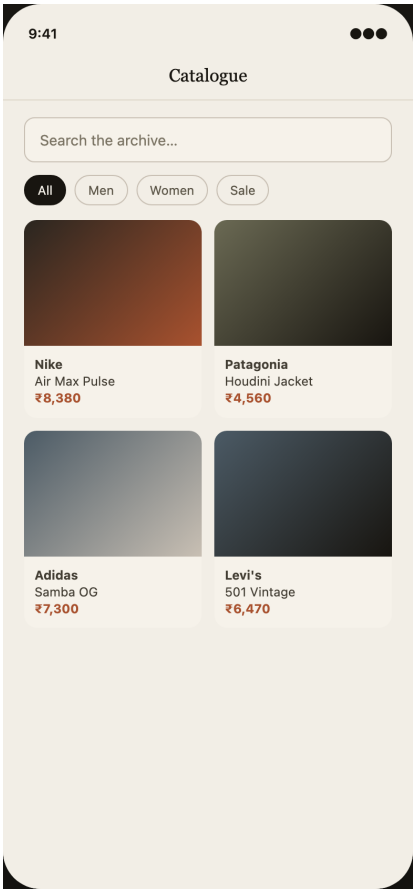


Figure 3: Catalogue cards from ProductCard props

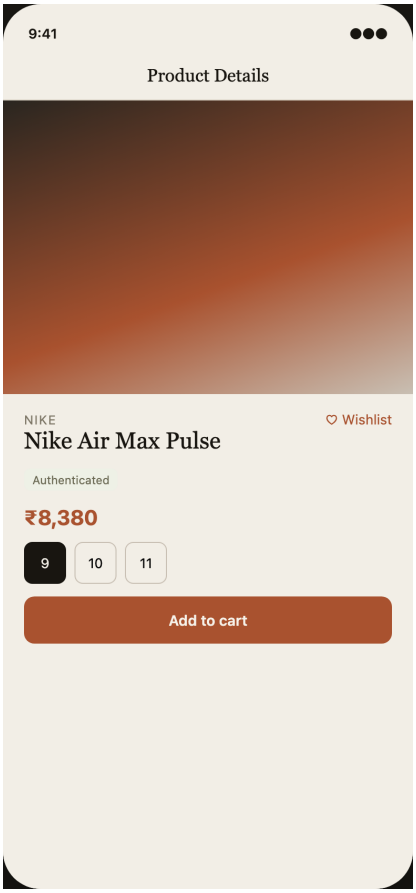


Figure 4: Product details — size state + wishlist

5. Assignment 4: Folder Structure and Reuse

5.1 Aim

To organise the project so each folder matches a course concept, with shared components reused across screens.

5.2 Documented folder structure

App.tsx / index.ts	Providers + RootNavigator entry
src/screens/	One screen module per feature
src/components/ui/	Reusable Button, Badge, Loader, EmptyState…
src/components/product/	ProductCard
src/context/	Auth, Cart, Wishlist, Profile, Orders
src/navigation/	RootNavigator (stack) + MainTabs
src/data/	products.json, collections.json
src/services/	Public API fetch + AsyncStorage
docs/	Academic write-ups and this submission

5.3 State in the application

- AuthContext — local register/login session (required for checkout).
- CartContext + useReducer with AsyncStorage persistence.
- WishlistContext and ProfileContext sync after sign-in.



Figure 5: ARCHIVE folder structure (Assignment 4)

6. Assignment 5: Customer Profile with Core Components

6.1 Aim

To implement a customer profile screen using the core React Native components required by the brief.

6.2 Core components used

View	Layout sections and navigation cards
Text	Name, labels, group titles
Image	Default profile avatar
TextInput	Name, email, phone, address, city, PIN
Button	Edit / Save profile (React Native Button)
Switch	Order notifications and newsletter
ScrollView	Full profile scroll container
StyleSheet	Bone/ink/clay theme tokens

6.3 Navigation list

Below the form, a settings-style navigation list pushes stack screens (Wishlist, Orders, Discover more, How we build, etc.) without cluttering the UI with raw buttons. Guests see Sign in / Create account; signed-in members see their account details.

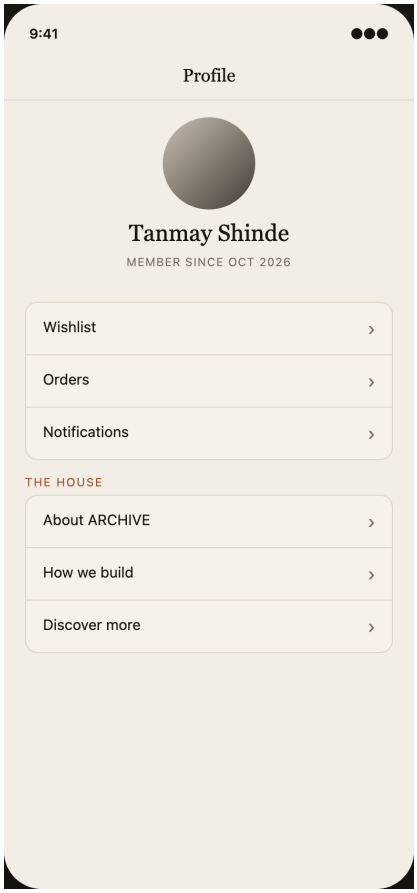


Figure 6: Customer Profile — core components + nav list (Assignment 5)

7. Assignment 6: Flexbox, TextInput and Events

7.1 Flexbox layouts

- Home quick actions and catalogue chips use flexWrap rows.
- Product size chips and cart lines use Flexbox rows with steppers.
- useWindowDimensions drives product gallery width and card columns.

7.2 Controlled inputs and onPress

- Login / Create account forms use controlled TextInput + validation Alerts.
- Checkout and Contact keep TextInput values in useState.
- Add to cart, wishlist, and Place order use Pressable/Button onPress handlers.

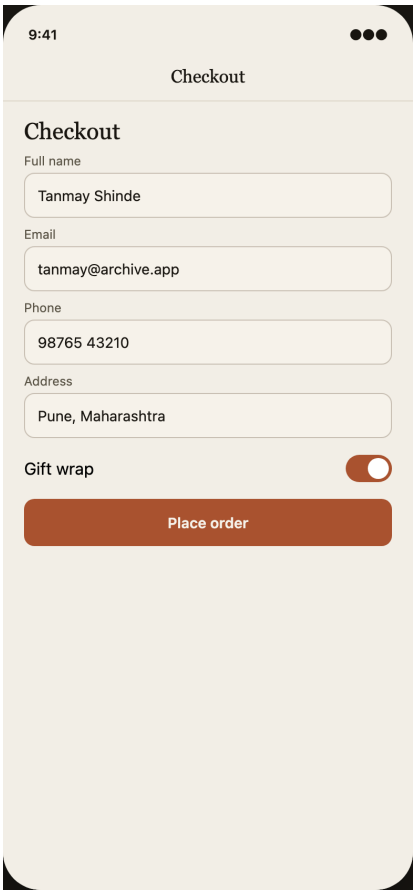


Figure 7: Checkout form — controlled inputs

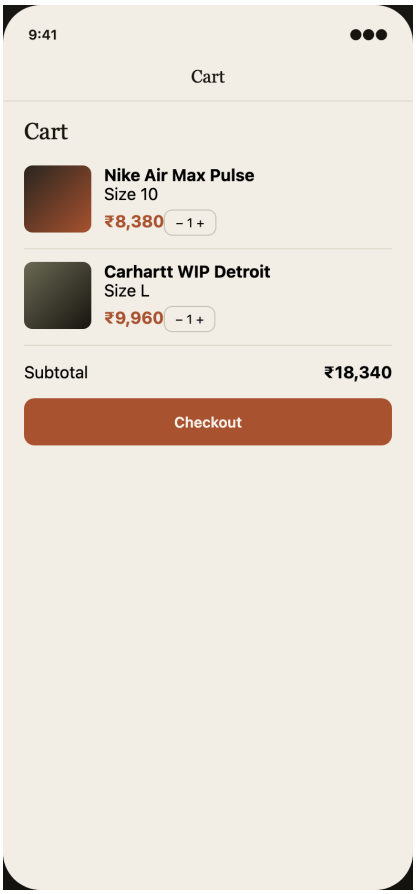


Figure 8: Cart — Flexbox rows + QuantityStepper

7.3 Sign in & create account

Guests may browse and add items to the cart. Placing an order requires a registered account. LoginScreen and RegisterScreen demonstrate TextInput, onPress validation, and navigation into Checkout after a successful session. Profile remains available after login with the member details from registration.

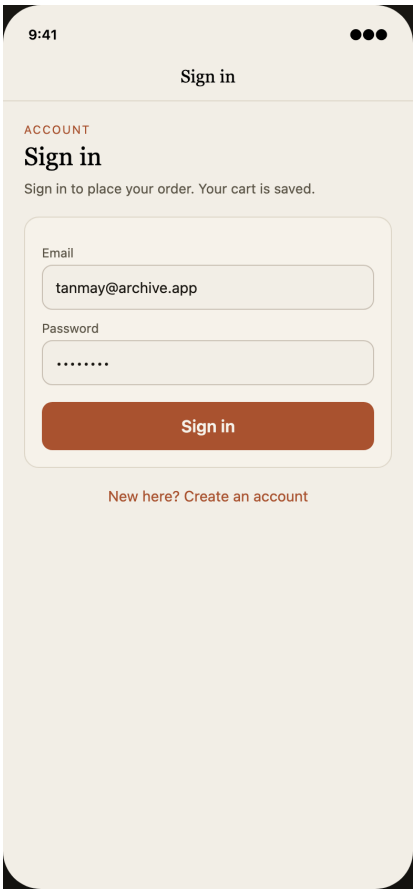


Figure 9: Sign in screen (Assignment 6)

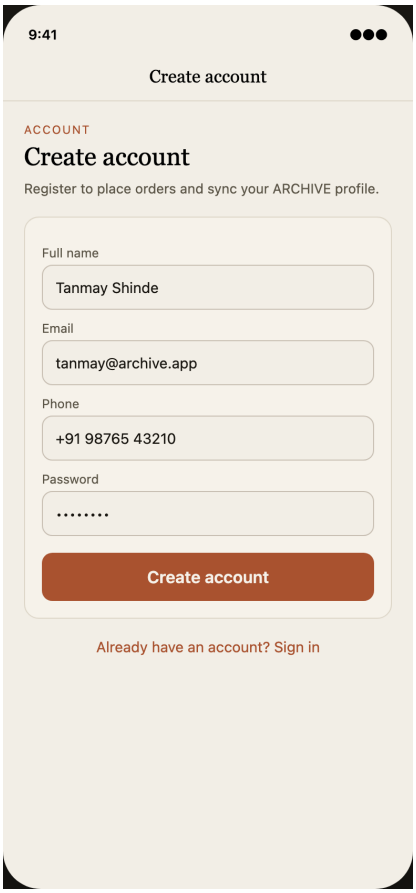


Figure 10: Create account screen (Assignment 6)

8. Assignment 7: Multi-Screen Navigation

7.1 / 8.1 Aim

To demonstrate multi-screen navigation with a mandatory Stack navigator and storefront chrome via bottom tabs.

8.2 Implementation

RootNavigator	Native stack: Hello, Product, Checkout, About, Discover, …
MainTabs	Bottom tabs: Home · Shop · Cart · Profile
Typed params	RootStackParamList / MainTabParamList (TypeScript)

Typical path: Home → Catalogue → Product Details → Cart → Checkout → Order Confirmation. Profile list items navigate into the same stack.

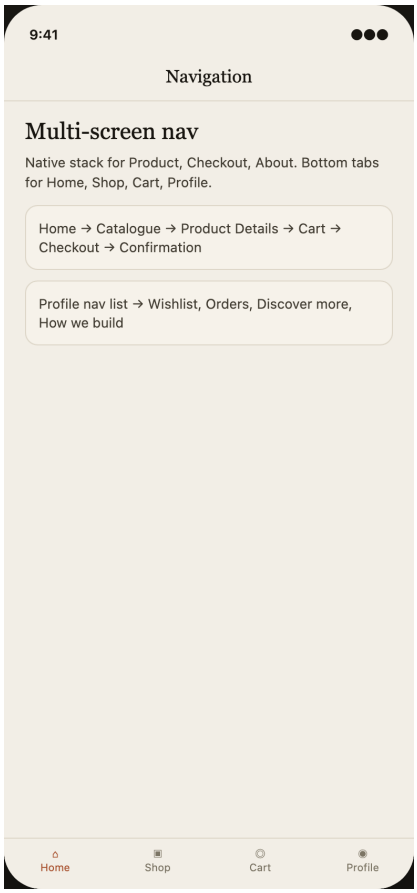


Figure 11: Bottom tab navigation bar (Assignment 7)

9. Assignment 8: Dynamic Data (JSON + Public API)

9.1 Local JSON FlatList

CatalogueScreen reads src/data/products.json (26 authenticated pieces, INR). FlatList uses numColumns, getItemLayout hints and FilterChip/search/sort for efficiency.

9.2 Public API demo

Discover more (ApiDemoScreen) fetches https://fakestoreapi.com/products with loading, error, retry and pull-to-refresh. Images are URI-encoded for Android.

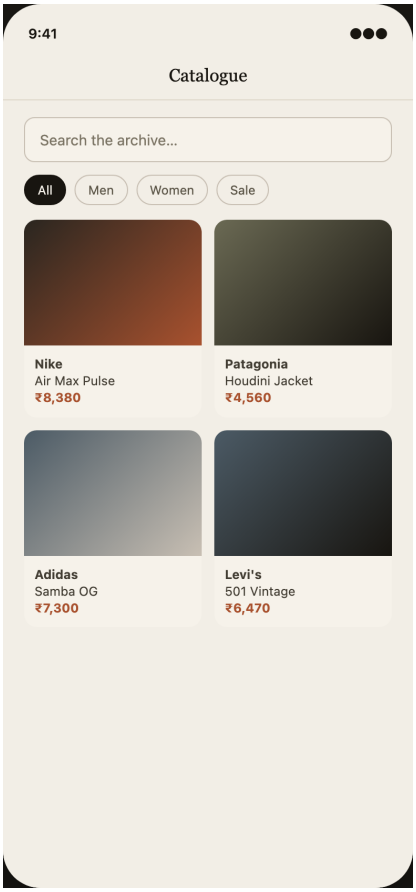


Figure 12: Local JSON catalogue FlatList

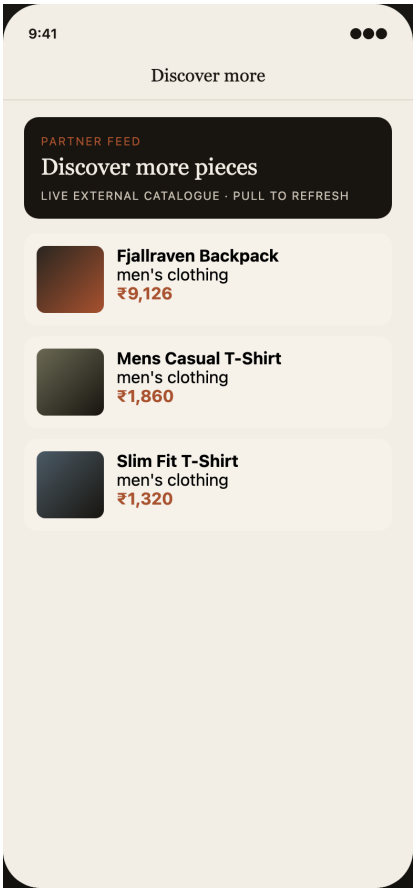


Figure 13: Public API partner feed

10. Assignment 9: E-Commerce Storefront

Assignment 9 requires a brand storefront. ARCHIVE delivers Home (hero + story), About, Authenticity & Services, Catalogue and Contact, linked through the stack navigator.

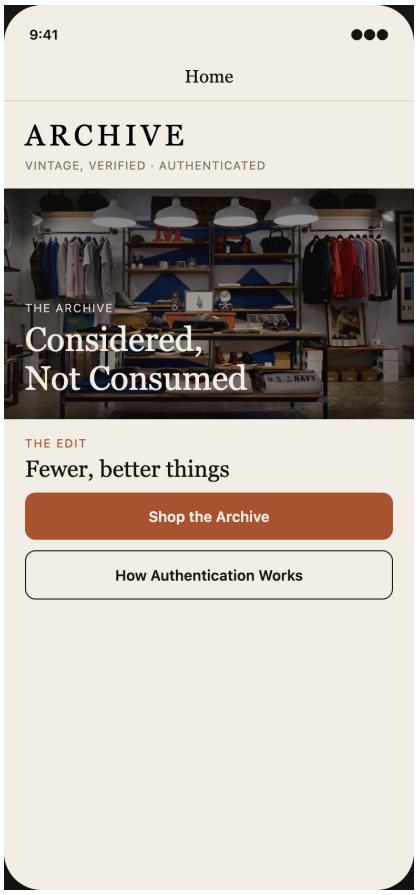


Figure 14: Home — brand hero and edit

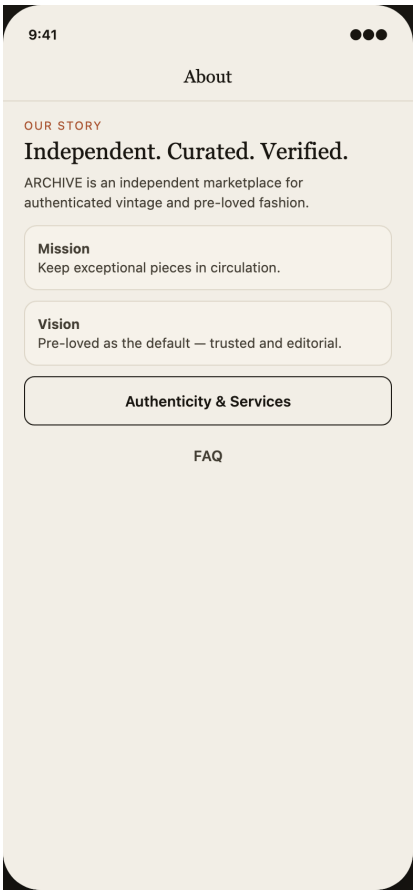


Figure 15: About ARCHIVE

11. Assignment 10: Complete Shopping Application

Assignment 10 is a strict superset of Assignment 9. ARCHIVE adds Product Details, Cart, auth-gated Checkout with validation, Customer Profile, search/filter/sort, wishlist and order confirmation — one Expo app, one stack.

- Product gallery, size selection, authenticity badge, add to cart.
- Cart persistence via AsyncStorage; quantity stepper; remove with Alert.
- Sign in / Create account required before Place order.
- Checkout validation → OrderConfirmation with generated order id.
- Profile + Preferences switches; account navigation list.

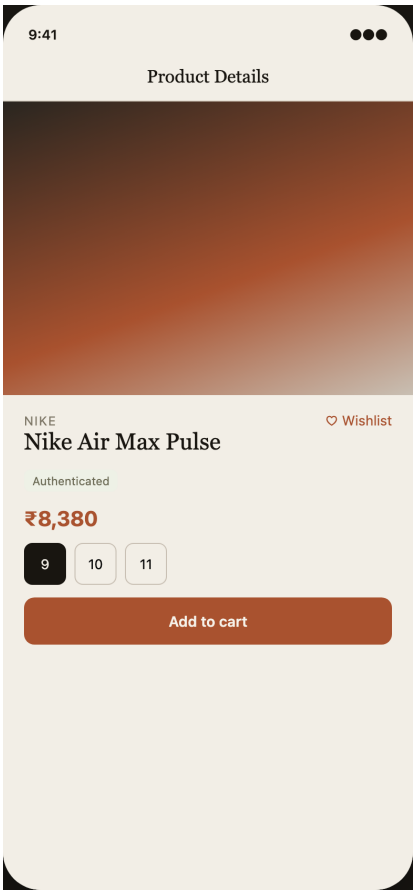


Figure 16: Product → Add to cart

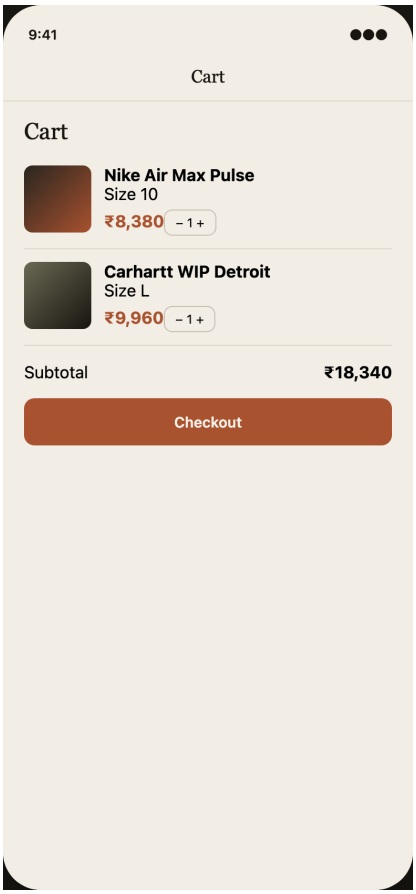


Figure 17: Cart → Checkout path

12. Rubric Alignment

The table below states where each evaluation criterion is demonstrated.

Functionality & Output	All screens navigable on Pixel_9; cart/checkout/API loading-error-retry; empty states.
Code Structure & Syntax	TypeScript strict, modular src/, ESLint/Prettier, typed navigators.
Concept Implementation	Props, useState/Context, core components, Flexbox, Stack+Tabs, FlatList, fetch, Alerts.
Readability & Documentation	README, docs/ASSIGNMENT_*.md, this report + presentation with screenshots.
Viva / Explanation	docs/VIVA_NOTES.md + How we build screen for comparative study.

13. Conclusion

Assignments 1 to 10 were completed as a single Expo React Native application — ARCHIVE — run on the Pixel_9 emulator. The project covers setup, comparative study, reusable components with Props and state, documented structure, core profile components, Flexbox and validated forms, stack + tab navigation, local JSON and public API lists, and a full shopping flow from catalogue to order confirmation. One idea summarises the design: a premium storefront UI that still maps cleanly onto every CPAD requirement.

Submitted by: Tanmay Shinde

Enrollment Number: ADT23SOCB1617

Project: ARCHIVE (https://github.com/tanmays0/ARCHIVE-APP)